Proactive Cache Optimization in Enterprise Systems Through Double Deep Reinforcement Learning

Jason Tanuwijaya Tariono
Computer Science Department, School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
jason.tariono@binus.ac.id

Deliko Hartono
Computer Science Department, School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
deliko.hartono@binus.ac.id

Reina Setiawan
Computer Science Department, School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
reina@binus.edu

***Abstract*—Cloud-hosted Enterprise Resource Planning (ERP) systems running on multi-tenant backends regularly hit latency problems once pushed past their load envelope. Conventional cache policies like Least Recently Used (LRU) act reactively, possessing no machinery to predict sequential business-logic workflows. The present study introduces a proactive caching middleware for supply-chain modules. A single Double Deep Q-Network (DDQN) agent observes an 18-dimensional state vector and selects from 11 discrete actions, fusing prefetches and eviction into one unified policy. Maximizing the agent's cache hit ratio directly minimizes expected query latency. Across twenty evaluation seeds on synthetic TPC-C workloads, the DDQN agent hits a 93.30% mean cache hit ratio (95% CI ±0.69%). It outpaces LRU by 5.01 percentage points, SLRU by 2.61 points, ARC by 4.15 points, and LFU by 0.51 points, while trimming mean analytical query latency by 40.47% against LRU. The advantage widens as memory tightens. At a tight two-slot configuration, the agent secures 40.43% against LRU's 15.39%. There is consistency as indicated by statistical testing (paired $t = 13.71$, $p < 0.000001$, Cohen's $d = 3.06$, Wilcoxon $W = 1.00$, $p = 0.000004$) which rules out the possibility of lucky initialization. One drawback of the results is that they are based on a single-threaded (sequential) driver and synthetic workloads. The achieved results can be used for the parallel concurrent multi-tenant traffic, which will be subject of further research.**

Keywords—proactive caching, deep reinforcement learning, Enterprise Resource Planning, prefetching, Multi-Tenant Architecture

I. Introduction

In the past couple of years, almost all deployments of Enterprise Resource Planning (ERP) have been cloud-based and multi-tenant. A configuration like this allows to use a single database backend for multiple concurrent requests in the tenant counts that can be in the thousands [1]. The layout is the one preferred by service providers in terms of cost. There, however, is a definite cost. As concurrent tenants perform read-intensive queries on the shared instance, contention builds up in the I/O path and the response time for each query starts to grow and in some combinations of workloads this can be observed within a fairly short time [2]. It is not just the latency that is associated with the user-facing layer. It propagates downstream into operational pipelines whose throughput depends on prompt return of data to the application stack.

The standard mitigation, in this regard, is to drop an in-memory cache between the application stack and the persistent storage tier, with Least Recently Used (LRU) eviction continuing to serve as the customary default replacement governor. The weakness most often noted about LRU is its reactivity: the policy only acts once a request has already landed, so an entry enters the cache only after a user has explicitly asked for it [3]. This reactive design becomes a real cost in ERP modules such as supply-chain management, where transactions almost always follow fixed sequences [4]. A sales order creation flow, for instance, almost always reaches for inventory stock data immediately after. Since the traditional replacement policy has no mechanism for anticipating that pattern, cold-start delays accumulate while the system waits on the database [2][3]. There is room here for a proactive caching layer that picks up these sequential patterns from the runtime signals themselves, instead of leaning on brittle hand-crafted heuristics taken from the general-purpose caching literature.

To close that gap, a Deep Reinforcement Learning (DRL) agent is installed as a proactive intelligent middleware layer between the application and the storage backend. While a conventional cache is waiting for a request, this type of agent does not. It knows by analyzing system signals, such as the history of accesses in the recent past and the current cache occupancy, in advance which data to load or remove from the cache [5]. This decision making process is simulated as an MDP in which the agent has to choose between 11 different actions at each time step, including specific prefetch and eviction actions. Both cache behaviours, traditionally handled by separate heuristics, are controlled within a single learned policy. Prefetching loads the data before the user needs it. Eviction removes entries that are unlikely to be accessed again soon [6]. The multi-tenant ERP setting is a natural application for this kind of learning agent, because the business logic that governs transactions already provides the predictable sequential structure that a reinforcement learning agent can exploit. The goal of this research is to reduce query latency in multi-tenant ERP systems through simulation. The experimental scope covers the supply-chain module with synthetic workloads built to mirror real sequential transaction patterns.

II. Related Work

Three lines of prior research line up directly with what comes next: tenant-level resource control under shared-instance databases, the wall that recency-and-frequency replacement hits inside enterprise traffic, and the recent imprint of reinforcement learning on database tuning together with prefetching. What sets multi-tenant ERP workloads apart from generic caching problems is that business-logic sequencing is structural, not statistical. Supply-chain transactions chain in near-deterministic order: a sales order creation is routinely followed by an inventory check, which typically precedes a delivery transaction [4]. Recency and frequency heuristics are blind to that chain, but a reinforcement learning agent can internalize the ordering directly from the runtime signals it observes.

A. Resource Management in Multi-Tenant Environments

Running many tenants against a single shared database is not a new problem, but it remains a difficult one. Resource contention and I/O spikes appear with very little warning once many users hit the same instance [7][8]. The typical response is tenant-aware allocation, distributing resources according to estimated demand [3][7]. They are all driven by fairly consistent traffic for an extended period of time. Due to traffic changes, the traditional allocation rules are not able to satisfy the changing traffic [3][7]. In the multi-tenant ERP space, very little discussion has been heard so far for responding at the runtime level, real-time per tenant.

B. Evolution of Traditional Cache Replacement Policies

LRU and LFU are backwards looking. The scores of each policy are based on what has already occurred: neither of the two scores will represent what the next transaction will be asking for [9]. The enterprise database queries are usually serial and not parallel, and are most often queries to the inventory database, and they are not frequent enough nor recent enough to capture the serial nature of the query [4][9]. The miss-rate curves of the two heuristics [10] both rise rapidly for large active working set sizes. The lower limit is not due to parameters being tuned, it is within the decision rule. There are two adaptive methods between purely reactive heuristic and learnt policies. The Adaptive Replacement Cache (ARC) maintains four lists: recentness, frequency, recent ghost list and ghost frequency list. It does an automatic balance between recency/frequency. SLRU works as LRU (Least Recently Used) with segmentation, it splits the cache into two regions: a probation region where new entry to the cache is added; a protected region, where entry is moved after the second hit to the cache. Both beat the standard LRU and LFU when working with this skewed workload, and can be used as a benchmark to compare them.

C. Deep Reinforcement Learning for Database Tuning

In previous studies, the database engine was already demonstrated to be well integrated with machine learning as shown below. Deep reinforcement learning has proven to be successful in configuration problems where there is direct interaction with the running system, unlike problems that can be packed into static traces, whose search space is too vast to be "tuned" with human effort [11]. Agents trained in that way are superior to experienced DBAs in workloads that require a lot of contention and are approximately 25% faster in their response times [5][11]. In the case of DQN, this is even more true because during runtime, it is given input from a combination of signals, and then returns an appropriate action [12] which is applied.

D. Advancements in Intelligent Prefetching Strategies

Prefetching aims to retrieve data before the request arrives. Older stride-based approaches worked for regular patterns, but modern applications access memory in far less predictable ways [13]. In multi-core settings, multi-agent RL coordinates prefetching across cores without creating bus congestion [6][13]. For ERP systems, what an effective prefetcher should track is the surrounding business logic. The canonical supply-chain flow proceeds from Sales Order through Inventory to Delivery. Agents trained inside partial-observability frameworks reproduce these chained transitions with higher fidelity than reactive counterparts [14], and once the policy has internalized those recurring sequences, the required entity sits in cache before the user reaches the next step [13][14].

E. Predictive Eviction Policies in High Traffic Systems

Just as much weight rests on eviction decisions as on prefetch decisions. Removing the wrong block from the cache wipes out the gain from a successful prefetch almost immediately. Recent cloud-storage research has applied DQN to rank cached entries by expected future value, with the lowest-ranking entries the first to leave [15]. Previous studies on this approach to predictive eviction have shown that the hit ratio of the cache can be improved by up to 18% over traditional predictive eviction policies during the peak hour since most of the information that will be referenced in the future is now in the cache longest [2][15][16]. All the literature surveyed has been about the assumption of admission and removal of cache as independent control problems, typically in CDN and cloud block storage domain, respectively, for sequential ERP traffic. This is addressed in the present work by employing a single DDQN agent that decides on both actions: an 18-dimensional state vector for both decisions is used, which is much simpler than having two separate policies, and is empirically stronger than all four baselines studied in the present work.

III. Material and Method

The objective was to design a modular back-end simulator that can simulate the performance of a workload for ERP supply-chain and a proactive caching approach to understand the performance gain in terms of query latency for the workload.

A. Proposed System Architecture

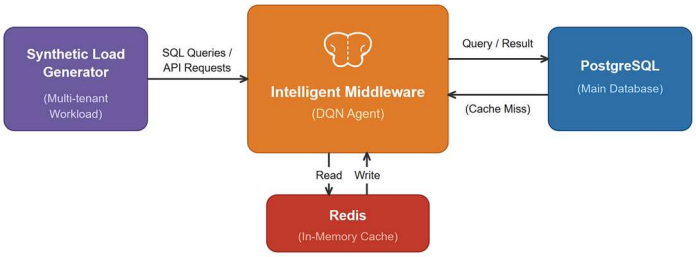


Fig. 1. High-level System Architecture of the Proposed Intelligent Caching Framework.

The simulator consists of three parts: workload generator to create multi-tenant SQL workload with business-logic patterns, intelligent middleware layer to execute the DQN agent and implement the agent to make all cache decisions, storage layer for keeping persistent database back-end (PostgreSQL) and in-memory cache back-end (Redis) [5][8]. The character of this three-layer dynamics is demonstrated qualitatively in Fig. 1. The architecture in this study is implemented in a sequential way across one thread while the architecture is designed to be deployed concurrently across multiple tenants. The reward signal is the one threaded driver with only the "pure" traceability to the policy and not any scheduling artifact. To evaluate for truly concurrent workload conditions, further studies will be dedicated to this.

B. DQN Agent Implementation

TABLE I. AGENT INPUT ENCODING: $S_t \in \mathbb{R}^{18}$ FEATURE BREAKDOWN

Index	*Feature*	*Description*
0–9	Query History	Last 10 accessed entity types (normalized)
10	Cache Occupancy	Current cache fullness ratio ($\frac{\|cache\|}{M}$)
11	Tenant Identity	Normalized tenant ID to identify active sessions
12–16	Cache Presence	Binary flags indicating if an entity type is in cache
17	Record ID	Last accessed record ID (normalized)

The framework contains one transition step per caching decision in the form of a Markov Decision Process [5]. On each iteration the agent reads off its environment as an 18-dimensional vector $S \in \mathbb{R}^{18}$. The next 10 positions (0-9) are used to represent the 10 most recent entity types accessed (normalized). Slot 10 is used to output information on how much of the cache is being used, slot 11 is used to store information about the currently active tenant session, and slots 12-16 are used to store binary flags representing which entity types are currently in the cache. The details of the complete schematic are provided in Table I.

The agent can choose from 11 different actions from the observed state. Actions 0-4 prefetch a certain entity type, actions 5-9 evict one of those, and action 10 does nothing if the cache has what it takes to pass on. Combining the two policies into a single shared policy makes it easier for the agent to learn the dynamics of both prefetch and eviction directly, instead of having to learn them separately by using two hand-tuned heuristics. There are 5 entity types in total. These are broken down into a transactional grouping of sales orders and inventory records, and an operational grouping of delivery schedules, customer information, and payment transactions. The entity set was chosen because it represents the canonical transaction mix defined by TPC-C benchmark [17] that is the standard reference workload for supply-chain access patterns. Reward signal for each step is binary. Returns $Rt = +1$ if the item is found, and $Rt = -1$ if the item is not found (miss case). Aggregated over an episode, the resulting signal acts as a direct proxy for the cache hit ratio (h), and pushing (h) higher is mathematically equivalent to driving the expected query latency $E[L]$ downward, expressed as:

$$E[L] = h \cdot L_{\{hit\}} + (1 - h) \cdot L_{\{miss\}} \quad (1)$$

The two penalty constants are pinned to the deployment stack. $L\{hit\}$ is fixed at 0.1 ms for a Redis cache hit, with $L\{miss\}$ fixed at 15.0 ms for a PostgreSQL fetch. It is worth noting that the gap between these two constants is what gives the cache hit ratio its leverage on overall query latency, corresponding to a PostgreSQL fetch. Since $L\{miss\} \gg L\{hit\}$, every additional percentage point of (h) cuts mean per-query latency by roughly the same magnitude. Fig. 2 sketches the agent-environment interaction loop in which the state is read, the action chosen, and the reward returned. The policy network relies on a two-layer fully connected structure. The 18-dimensional state vector feeds into two hidden layers of 64 units each with ReLU activations, terminating at 11 output units for per-action Q-value estimates. The target network mirrors this exactly. Total trainable parameters across both networks were kept to 6,091. This keeps inference overhead well below the 0.1 ms Redis fast path, making CPU-only training actually practical. Weight updates are handled by the Adam optimizer. Compact parameter count means that there is a reduced risk of overfitting because a larger network simply learns noise. The following four baseline designs are considered: LRU, LFU, ARC, and SLRU (described in Section II.B) All with the same Redis and PostgreSQL connections as the DDQN agent.

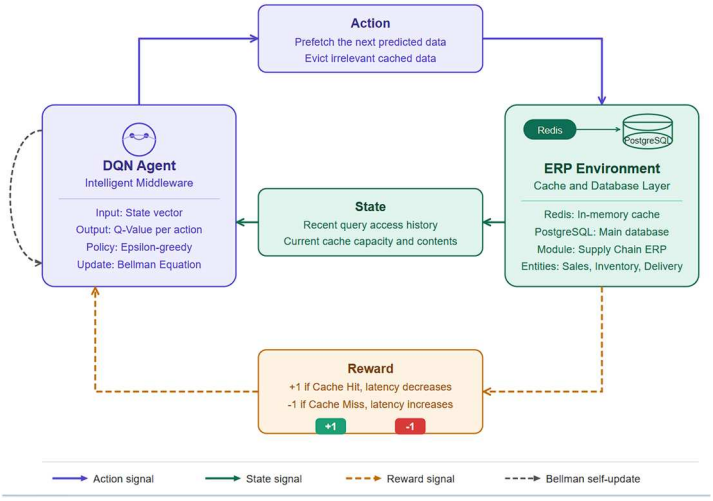


Fig. 2. Interaction Loop Connecting the Agent to the Caching Environment.

C. Q-Value Update and Learning Rule

A vanilla DQN training loop will cause the Q-value to tend to increase over time, which will cause the learned policy to get worse over time. A Double Deep QNetwork (DDQN) variant is used, with a pair of networks for action selection and action evaluation, to reduce the drift [18]. Side by side are then two networks. The action choice to take is passed to the policy network with parameters (θ). The selected action is then rated by the target network parameterized as (θ-). This way of splitting the two networks results in the target for the temporal-difference as:

$$y_t = r_t + \gamma \cdot Q(s_{\{t+1\}}, \arg\max_a Q(s_{\{t+1\}}, a; \theta); \theta^-) \quad (2)$$

Every 10,000 training steps, the target network is copied from the live network in one update, meaning that all policies are updated in sync. Gradient norms are bounded to avoid blow-up which can occur during the early training phase. Training stops when there is clear evidence the loss curve has levelled off. The value at convergence is about 0.10, which is expected. The complete set of hyperparameters used in all experiments is shown in table II.

TABLE II. HYPERPARAMETER CONFIGURATIONS

Parameter	*Value*
Learning Rate (α)	0.001
Discount Factor (γ)	0.95
Batch Size	64
Replay Memory Capacity	50000
Exploration Rate (ϵ)	Decays from 1.0 to 0.01

D. Synthetic Workload Generation

Unfortunately, because of privacy concerns, real production ERP logs were not available for use as evaluation data. Instead, the traffic was generated from access patterns specified in the TPC-C benchmark [17]. The traffic intensity of tenants is distributed according to a Pareto distribution with α = 2.0, meaning that most of the traffic is directed toward a small group of tenants. This concentration on the tails is what real enterprise systems have when they are running in steady state. A business-logic ordering is added to the base

distribution by adding Markov-chain transitions to the synthetic stream. There are examples of how a Sales Order request might roll up into an Inventory or Delivery query, in which case 70% of the time, the same ordering relationship would be reflected in the synthetic stream as well, avoiding the need to recreate that relationship from scratch. The resulting traffic, taken as a whole, supplies a reasonable proxy for real multi-tenant ERP load, including the bursty arrival patterns that real enterprise environments routinely exhibit [10][17].

E. Real ERP Workload Validation

An additional robustness scenario draws entity weights from 7,260 transaction records collected from a production Odoo 14 deployment: Products 9%, Sales 31%, Invoices 18%, Company 26%, Assets 16%. The agent is tested on this Finance/CRM-dominant profile without retraining, so the result directly measures zero-shot transfer from the supply-chain training domain. Results appear in Section IV.D.

IV. Result and Analysis

All latency figures reported in this section are derived analytically against fixed access penalties of 0.1 ms per Redis hit and 15.0 ms per PostgreSQL miss, an arrangement that, by construction, isolates the comparison to the quality of caching decisions alone, with hardware variability and network-induced jitter held outside the picture.

A. Overall Cache Hit Ratio Performance

Across twenty evaluation seeds, the DDQN agent hits a 93.30% mean cache hit ratio (95% CI ±0.69%). It ranks first among all five policies in 17 of 20 seeds, maintaining its lead over LRU in 19 of 20 seeds. LFU trails at 92.79% ± 0.15%, SLRU at 90.69% ± 0.20%, ARC at 89.15% ± 0.26%, and LRU at 88.29% ± 0.25%. There is one catch: seed 9. That initialization produces a DDQN hit ratio of 87.52%, falling below all four baselines due to a locally suboptimal policy initialization. Excluding seed 9, the mean rises to 93.60% and the policy ordering holds without exception. The source of the advantage comes down to what each policy actually learns. LRU scores entries by recency, LFU by frequency. Neither cares about what the next step in a transaction will request. The DDQN agent, on the other hand, reads the Sales Order → Inventory → Delivery transition signal directly from its 18-dimensional state vector. It prefetches the next likely entity before the user even triggers it. This mechanism produces a 5.01-percentage-point lead over LRU, a 4.15-point lead over ARC, a 2.61-point lead over SLRU, and a 0.51-point lead over LFU.

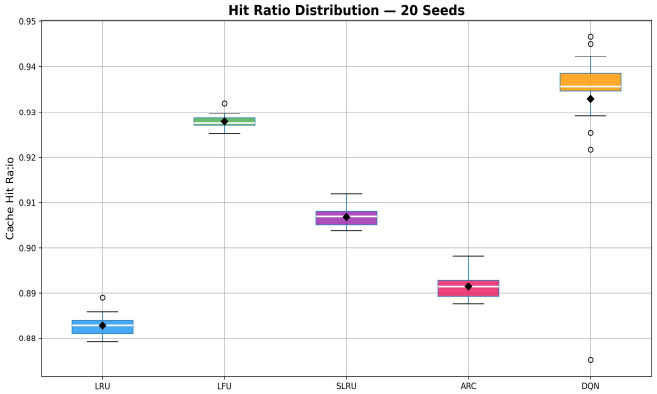


Fig. 3. Cache Hit Ratio Distribution Across Twenty Evaluation Seeds for All Five Policies

B. Query Latency Reduction and Stability Analysis

With cache miss cost fixed at 15.0 ms against 0.1 ms per hit, the analytical mean query latency for the DDQN agent resolves to 1.10 ms, against 1.85 ms for LRU (40.47% reduction), 1.49 ms for SLRU (26.17% reduction), 1.72 ms for ARC (36.02% reduction), and 1.17 ms for LFU (6.45% reduction). These figures isolate caching decisions alone, holding hardware variability outside the measurement. A supplementary stochastic run, where latency is sampled from log-normal distributions centred on the same hit and miss constants with additive jitter drawn from an exponential distribution, yields DQN mean latency of 1.25 ms against LRU's 2.11 ms, preserving the policy ranking and confirming the latency advantage is not an artifact of the fixed-penalty model. The drop is visible directly in the per-query trace in Fig. 4, which is flat for DQN after a brief exploration spike at the start of the test set. Beyond the mean, per-seed latency standard deviation under DDQN averages 4.60 ms versus 5.99 ms for LRU (23.1% less spread), with the 99th percentile at 23.26 ms against LRU's 26.80 ms (13.2% tail reduction). Most residual variance comes from the cold-start exploration window at the start of each run. The runtime cost of the learned policy was measured across all 20 seeds. Inference runs at 0.073 ms per step, well below the 0.1 ms Redis fast path and the 6,091-parameter model trains in roughly 14 minutes per seed on CPU, peaking at 442 MB of memory. These numbers sit comfortably inside typical server resource budgets.

TABLE III. COMPUTATIONAL OVERHEAD SUMMARY

Metric	***Value***	***Unit***
Inference time (train/eval)	0.073 ± 0.008 / 0.073 ± 0.016	ms/step
Gradient update time	2.08 ± 0.44	ms/step
Peak memory	441.98	MB
Training wall-clock	861.1	seconds

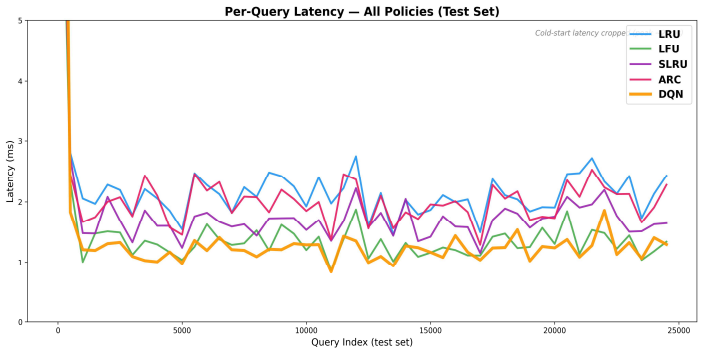


Fig. 4. Per-Query Analytical Latency: DDQN Agent Versus Four Baselines Over the Test Set

C. Cache Size Sensitivity Analysis

Eight cache sizes were tested, spanning 2 slots up to 20. The pattern in Table IV and Fig. 5 inverts the obvious intuition: DQN's lead grows as memory shrinks, and at the two-slot end the DDQN agent holds 40.43% while LRU manages 15.39%. This is a 25-percentage-point gap. It also comfortably beats the best adaptive baselines (ARC at 19.26%, SLRU at 21.74%) by margins of 21 and 19 points respectively. At twenty slots the five policies cluster: 93.55% for DQN, 92.65% for LFU, 90.43% for SLRU, 88.84% for ARC, and 87.91% for LRU.

The two-slot regime is special. There are no spare buffers to absorb recovery cost associated with wrong evictions, which become permanent. LRU does not know what to expect, the miss already happened by the time of eviction. The DDQN agent gets around this by fixing the next probable entity in its

state vector before the user has arrived. That edge is reduced as capacity eases, but it's in that memory-confined environment when enterprise tenants are most common that the learned policy pays off best.

TABLE IV. CACHE HIT RATIO ACROSS CACHE SIZES

Cache Size	*LRU Hit*	*LFU Hit*	*SLRU Hit*	*ARC Hit*	*DQN Hit*
2	15.39%	24.01%	21.74%	19.26%	40.43%
4	34.83%	53.67%	49.98%	43.60%	61.75%
6	51.44%	68.59%	66.04%	59.36%	73.86%
8	63.09%	76.23%	73.74%	68.51%	80.52%
10	70.98%	81.22%	77.80%	74.54%	84.63%
12	76.30%	84.70%	81.94%	78.96%	87.75%
16	83.43%	89.64%	87.18%	84.85%	91.18%
20	87.91%	92.65%	90.43%	88.84%	93.55%

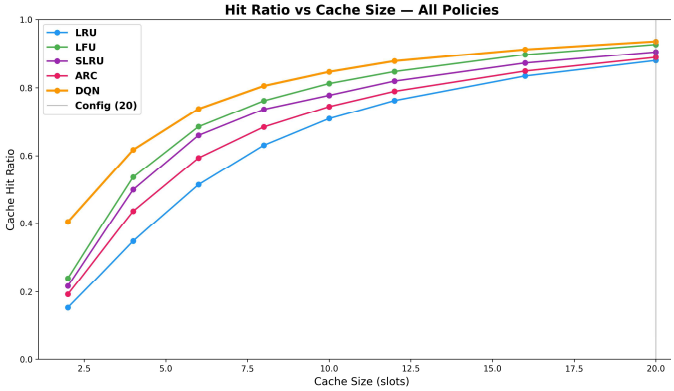


Fig. 5. Cache Hit Ratio Versus Cache Size: DDQN Agent Versus Four Baselines

D. Statistical Significance and Robustness

This is supported by the numbers in all four comparison pairs. Against LRU, results show $t = 13.71$, $p < 0.000001$, Wilcoxon $W = 1.00$ ($p = 0.000004$), and a seed-level Cohen's $d = 3.06$. SLRU gets $t = 7.23$ ($p = 0.000001$) and ARC hits $t = 11.17$ ($p < 0.000001$). A 95% confidence interval for the mean hit ratio of the DDQN was found to be ±0.69%. Even the minimum possible policy, the policy advantage over LRU stays above 4.3 percentage points. Two Cohen's d values are presented. The 3.06 value of d obtained from 20 paired seed means is more credible than the inflated value of 12.84 obtained in a 5 seed run because the larger the number of seeds involved, the more variance is exposed. The per-query d is 0.16, which is moderate, meaning that the majority of queries in particular are resolved correctly, irrespective of their policy. Both figures should be used with a certain degree of caution; the fixed Pareto shape ($\alpha = 2.0$) represents the suppression of sampling noise, and the deterministic Markov matrix represents the low-variance workload design as well as the strength of the policy. Per-seed hit ratios span 87.52% to 94.66%. The DDQN agent's performance is worse than LRU only in Seed 9. If that seed is tossed out, the range is 92.17%–94.66%, and the policy beats all four baselines without exception. This is further reflected in the training trace. The DDQN loss curve falls from 0.50 to below 0.15 within 18,000–38,000 gradient steps out of 198,750 total, then plateaus. The hit ratio rises steadily for all 20 seeds, rising from 55% at epoch 1 through to 93% at epoch 8. 0.09 is a statistical loss due to the nature of the Pareto workload and not to the lack of training.

Sweep of learning rate across three values is a confirmation of stability. With the default $\alpha = 0.001$, the hit ratio is converging to 94.62% after about 38,000 gradient steps. Slowing to $\alpha = 0.0001$ lands slightly higher at 94.88% but takes 50,766 steps. High learning rates cause Q-value oscillation in this environment; with $\alpha = 0.01$ (aggressive) the hit ratio dropping to just 75.85%, which is completely unstable for training. A trained agent was used as an agent without retraining in nine synthetic configurations (three transition profiles x three Pareto alphas) and one real-ERP scenario. The agent outperforms LRU by +3.8 to +14.3 pp across all synthetic configs, with the highest gain in the Manufacturing/Logistics workload configurations with a small degree of tenant concentration ($\alpha = 1.5$). The real-ERP scenario (Odoo 14, 7,260 records; Section III. E) reverses the ranking: DQN records 86.13%–86.69%, trailing all baselines including LRU at 86.99%–87.15%. The supply-chain policy prefetches sequences that are not part of the Finance/CRM workload; this results in cache pollution instead of in useful anticipatory loading. Outside training domain, retraining should be provided for modules.

TABLE V. ROBUSTNESS SWEEP: DQN DELTA VS LRU (SELECTED CONFIGURATIONS)

Configuration	*Module Type*	*α*	*DQN Hit*	*DQN – LRU (pp)*
high_seq	Manufacturing/Logistics	1.5	90.00%–90.46%	+13.6 to +14.3
mid_seq	Supply Chain (training)	2.0	93.27%–93.52%	+4.99 to +5.53
low_seq	HR/Admin	2.0	92.42%–92.59%	+3.84 to +3.92
real_erp	Real ERP-Informed (Odoo 14)	2.0	86.13%–86.69%	−0.46 to −1.02

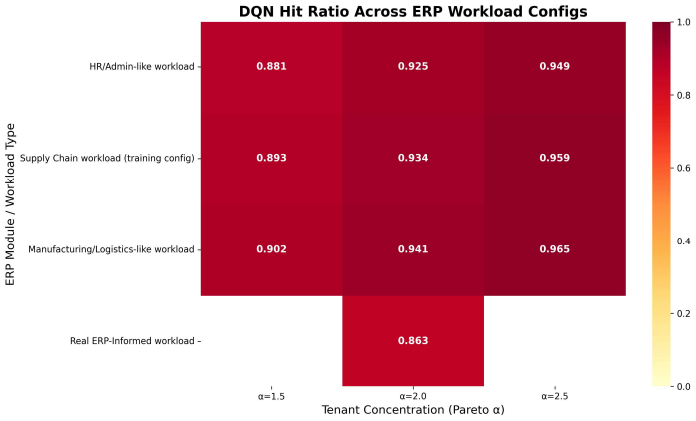


Fig. 6. Robustness Sweep: DDQN Hit Ratio Advantage Over LRU Across ERP Workload Configurations

V. CONCLUSION

It is in the multi-tenant ERP where query response time is most consistently the operational pressure point which emerges [19]. Reactive schemes like LRU can only act when the request has been already received, meaning that they don't know what the next step is in an in-flight transaction. In this regard, the present research proposes a proactive caching middleware in which the prefetch and eviction decisions are taken by a single DDQN agent operating in one common policy. On 20 seeds, the trained DDQN agent achieves a mean 93.30% cache hit ratio (95% CI ±0.69%). The agent improves LRU by 5.01 percentage points, improves SLRU by 2.61 points, improves ARC by 4.15 points and improves LFU by 0.51 points. Those costs, per hit and per miss, result in huge latency reductions: 40.47% reduction over LRU, 26.17% reduction over SLRU, 36.02% reduction over ARC, and 6.45% reduction over LFU. This policy ranking is also stable

in the presence of stochastic latency. The picture becomes clearer with the cache-size sweep: as the cache budget gets smaller, the relative advantage widens, to the point that the difference hits a 25-percentage point chasm at the two-slot setting (40.43% vs. 15.39%). A paired t-test across 20 seeds yields $t = 13.71$ ($p < 0.000001$), with a seed-level Cohen's d of 3.06 backed by Wilcoxon $W = 1.00$ ($p = 0.000004$). None of those numbers are close calls. Three caveats apply. All experiments run on a sequential single-thread driver, which isolates the caching policy cleanly but leaves open whether the same gains survive under concurrent tenant traffic. Latency values, while confirmed under stochastic simulation, still derive from fixed per-operation constants rather than live infrastructure measurements. The workload is entirely synthetic: as the real-ERP robustness sweep in Section IV.D shows, performance drops below LRU when entity access patterns depart from the training distribution, confirming the policy is supply-chain-specific and that module-level retraining is needed elsewhere. Two natural extensions follow from these caveats: a rerun of the same agent inside a concurrent multitenant workload [20] to find out whether the residual exploration noise climbs after shared scheduling joins the loop, then bringing in Dueling DDQN with Prioritized Experience Replay [21] in place of the plain DDQN core to find out whether the residual training loss reflects an architectural ceiling rather than a workload-bound floor.

ACKNOWLEDGMENT

The real-ERP transaction traces used in Section III.E were drawn from a production Odoo 14 deployment accessed through an industry internship arrangement. We declare that we used Claude in this manuscript to check for grammar and paraphrasing. The result from the AI was not used directly and was under human supervision for error checking.

OPEN DATA AVAILABILITY

This research did not rely on any open or publicly available dataset. All data can be accessed at: *https://github.com/Okiled/DDQN-ERP-Cache/tree/main.*

AUTHOR CONTRIBUTION STATEMENT

Jason Tanuwijaya Tariono: conceived the initial research idea, worked out the preliminary structure of the DDQN model, supporting literature citations. **Deliko Hartono:** wrote the full DDQN implementation from scratch and ran the training experiments. **Reina Setiawan:** validated the initial research direction, offered on how the manuscript could be strengthened further.

REFERENCES

[1] J. G. Costa et al., "Multitenancy in single instance of data persistence service for supporting low-code platforms," IEEE Access, vol. 13, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10824762

[2] Y. Lu et al., "MTRP: A high-performance cost-efficient buffering scheme for multi-tenant cloud services," in Proc. 2024 IEEE 30th Int. Conf. Parallel Distrib. Syst. (ICPADS), 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10885099

[3] Y. Zhang et al., "Agile cache replacement in edge computing via offline-online deep reinforcement learning," IEEE Trans. Parallel Distrib. Syst., vol. 35, no. 4, pp. 663–674, Feb. 2024, doi: 10.1109/TPDS.2024.3368763.

[4] J. Wang et al., "Real-time workload pattern analysis for large-scale cloud databases," Proc. VLDB Endowment, vol. 16, no. 12, pp. 3689–3701, 2023, doi: 10.14778/3611540.3611557.

[5] B. Yin et al., "An efficient deep reinforcement learning-based automatic cache replacement policy in cloud block storage systems," IEEE Trans. Comput., vol. 73, no. 1, pp. 120–134, Jan. 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10288208

[6] C. Block, G. Gerogiannis, and J. Torrellas, "Micro-MAMA: Multi-agent reinforcement learning for multicore prefetching," in Proc. 58th IEEE/ACM Int. Symp. Microarchitecture (MICRO), 2025.

[7] C. Batista, B. Proença, E. Cavalcante, T. Batista, F. Morais, and H. Medeiros, "Towards a multi-tenant microservice architecture: An industrial experience," in Proc. 2022 IEEE 46th Annu. Comput., Softw., Appl. Conf. (COMPSAC), 2022, pp. 553–562, doi: 10.1109/COMPSAC54236.2022.00100.

[8] M. K. Ghali, A. Farrag, D. Won, and Y. Jin, "Database systems in the big data era: Architectures, performance, and open challenges," IEEE Access, vol. 13, 2025. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=11008610

[9] H. J. Yang, J. Fang, and M. Cai, "A prefetch-adaptive intelligent cache replacement policy based on machine learning," J. Comput. Sci. Technol., vol. 38, no. 2, pp. 391–404, 2023, doi: 10.1007/s11390-022-1573-3.

[10] B. Feng and Z. Ding, "Application-oriented cloud workload prediction: A survey and new perspectives," Tsinghua Sci. Technol., vol. 30, no. 1, pp. 34–54, Feb. 2025, doi: 10.26599/TST.2024.9010024.

[11] X. Zhao, X. Zhou, and G. Li, "Automatic database knob tuning: A survey," IEEE Trans. Knowl. Data Eng., vol. 35, no. 12, pp. 12470–12490, Dec. 2023, doi: 10.1109/TKDE.2023.3266893.

[12] Liu, X., et al. (2023). HWTune: A hardware-aware database tuning system with deep reinforcement learning. *2023 IEEE International Conference on Web Services (ICWS)*. https://ieeexplore.ieee.org/document/10294845

[13] F. Souza and D. Freitas, "Reinforcement learning-based cache replacement policies for multicore processors," IEEE Access, vol. 12, pp. 79177–79188, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10547236

[14] J. Kim, Y. Sun, and M. Huh, "Miss-triggered content cache replacement under partial observability: Transformer-decoder Q-learning," Mathematics, vol. 13, no. 19, p. 3217, 2025, doi: 10.3390/math13193217.

[15] Y. Luan, F. Sun, and J. Zhou, "A service-caching strategy assisted by double DQN in LEO satellite networks," Sensors, vol. 24, no. 11, p. 3370, 2024, doi: 10.3390/s24113370.

[16] S. Yun, D. Kim, S. Lee, and J. Lee, "i-CU: Intelligent cache replacement and content update for data freshness in cloud-edge networks," IEEE Trans. Mobile Comput., vol. 24, no. 12, pp. 12742–12755, Dec. 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11081942

[17] Transaction Processing Performance Council, TPC Benchmark C (TPC-C) Specification, Version 5.11, 2024. [Online]. Available: http://www.tpc.org/tpc_documents_current_versions/pdf/tpc-c_v5.11.0.pdf

[18] J. Yan et al., "Double deep Q-network based joint edge caching and content recommendation with inconsistent file sizes in fog-RANs," IEEE Trans. Veh. Technol., vol. 73, no. 3, pp. 4264–4276, Mar. 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10301684/

[19] D. T. Fagbore, O. N. Oyelade, and A. A. Oyetunji, "Performance trade-offs in multi-tenant IoT-cloud security: A systematic review of emerging technologies," Mach. Learn. Knowl. Extr., vol. 7, no. 1, p. 21, 2026. [Online]. Available: https://www.mdpi.com/2624-831X/7/1/21

[20] H. Li, J. Wang, and M. Chen, "Proactive caching with DRL in cloud-edge assisted cyber-physical-social systems," in Proc. 2023 IEEE Int. Conf. Dependable, Autonomic Secure Comput. (DASC), 2023, doi: 10.1109/DASC/PiCom/CBDCom/Cy59711.2023.10361338.

[21] P. Zhang, S. Li, D. Li, Q. Ding, and L. Shi, "Sensor-generated in situ data management for smart grids: Dynamic optimization driven by double deep Q-network with prioritized experience replay," Appl. Sci., vol. 15, no. 11, p. 5980, 2025, doi: 10.3390/app15115980.